
Un Apunte de

Deep Learning

Antonio Jara Sánchez

Versión: 24 de agosto de 2026

Prefacio

Este apunte, como el intento de presentar los temas del curso de forma progresiva que es, fue realizado para fines de estudio personal de la asignatura **DLY0100**. Se basa casi en su totalidad en el material del curso (diapositivas), así como notas de clase.

Esta es la versión del 24 de agosto de 2026. Cualquier sugerencia, corrección, observación, etc. puede ser indicado al correo antoniojarasanchez@proton.me

Índice general

Prefacio	i
1 Introducción al Deep Learning	1
1.1 Contexto tecnológico: la Sociedad 5.0	1
1.2 Inteligencia Artificial: definición y evolución	1
1.2.1 Clasificaciones de la Inteligencia Artificial	2
1.3 Machine Learning: el núcleo de la IA	2
1.4 Deep Learning: aprendizaje por capas	3
1.4.1 Machine Learning vs Deep Learning	3
1.4.2 Cómo trabaja Deep Learning	3
1.5 Aplicaciones del Deep Learning	4
1.5.1 Visión por computadora (Computer Vision)	4
1.5.2 Procesamiento de Lenguaje Natural (NLP)	5
1.5.3 Series temporales	5
1.5.4 Deep Learning generativo y GANs	6
1.6 Ética, sesgo y regulación	6
1.6.1 Sesgos discriminatorios	6
1.6.2 Privacidad y regulación	6
2 El Perceptrón y Redes Fully Connected Feed Forward	7
2.1 De la IA a las redes neuronales	7
2.2 El perceptrón	7
2.2.1 Operación del perceptrón	8
2.2.2 Funciones de activación	10
2.3 Arquitectura de una red neuronal	11
2.3.1 Multi-perceptrón	11
2.3.2 Redes Feed Forward Fully Connected (FFFC)	12
2.3.3 Función de salida: Softmax	13
2.3.4 Entrenamiento de la red neuronal	14

Capítulo 1

Introducción al Deep Learning

1.1 Contexto tecnológico: la Sociedad 5.0

En 2015, Japón acuñó el concepto de **Sociedad 5.0**, cuya idea central es que *las máquinas serán las principales creadoras del conocimiento humano*, apalancadas en la inteligencia artificial. Mientras la Sociedad 4.0 generó la información digital, la 5.0 la aprovecha con IA para integrar la tecnología en la vida cotidiana.

1.2 Inteligencia Artificial: definición y evolución

Se adopta la definición de Nilsson (1998):

Definición 1.1:

Inteligencia Artificial (IA): capacidad de imitar comportamientos inteligentes.

La definición de IA ha evolucionado a lo largo del tiempo.

Cuadro 1.1: Evolución de la definición de Inteligencia Artificial.

Definición histórica	Idea central	Problema / observación
Sistemas que piensan como humanos	Imitar el pensamiento humano	No existe una única forma de pensamiento humano
Sistemas que piensan racionalmente	Pensar de manera correcta	El silogismo carece de contexto
Sistemas que actúan como humanos	Test de Turing	No tuvo éxito en su época por capacidad de procesamiento
Sistemas que actúan racionalmente	Actuar según lo programado	Definición vigente desde 1998

Definición 1.2:

IA (definición vigente): todo sistema capaz de *actuar racionalmente*.

Racional: correcto para lo que fue programado; no implica conciencia.

1.2.1 Clasificaciones de la Inteligencia Artificial

Cuadro 1.2: Clasificación de la IA según capacidad y complejidad.

Tipo	Descripción	Ejemplos
IA reactiva	No aprende de experiencias pasadas; responde a estímulos actuales	Alexa, Siri, aspiradora robot, Deep Blue
IA con memoria limitada	Aprende de datos históricos, pero con memoria limitada y específica	Vehículos autónomos, Waze, Google Maps

Cuadro 1.3: Clasificación de la IA según propósito y aplicaciones.

Tipo	Descripción	Ejemplos
IA débil / narrow	Específica para tareas claras; no aprende tareas nuevas	Detector de spam de Gmail, asistentes virtuales, sistemas de recomendación, reconocimiento de voz
IA fuerte	Capaz de aprender cosas nuevas	Autos autónomos, sistemas de Machine Learning
Super IA	Consciente de su existencia; capaz de realizar cualquier tarea humana	No existe todavía

1.3 Machine Learning: el núcleo de la IA

Definición 1.3:

Machine Learning (aprendizaje automático): campo de la IA que se encarga de reconocer patrones desde los datos y de generalizar conocimiento a partir de la experiencia y de muchos datos.

Definición de **Arthur Samuel**: “el diseño y desarrollo de algoritmos que permiten a los computadores *aprender sin ser explícitamente programados*”.

No existe un comando “.learn”, el aprendizaje se logra mediante un **algoritmo**.

Definición 1.4:

Algoritmo: secuencia lógica, paso a paso, con inicio, decisiones y fin u objetivo.

Cuadro 1.4: Tipos de aprendizaje automático.

Tipo	Descripción	Ejemplos
Supervisado	Aprende con ejemplos etiquetados	Detección de spam, reconocimiento de voz, analíticas de anuncios
No supervisado	Encuentra patrones sin etiquetas previas	Sistemas de recomendación, registros de conductas, detección de anomalías
Semi-supervisado	Mezcla de supervisado y no supervisado	Análisis de voz, detección de fraude, investigaciones médicas
Por refuerzo	Aprende mediante prueba, error y refuerzo	Robótica, videojuegos, gestión de recursos, IoT

1.4 Deep Learning: aprendizaje por capas

Definición 1.5:

Deep Learning (aprendizaje profundo): sistema capaz de aprender mediante algoritmos y *sin intervención manual* de un usuario.

Definición 1.6:

Representación: forma en que los datos son transformados y procesados a lo largo de las capas para extraer características relevantes de manera automática.

Profundidad del modelo: cantidad de capas sucesivas de representación.

Advertencia 1.1:

“Profundo” *no* significa comprensión más profunda; significa *muchas capas sucesivas de representación*.

¿Por qué despegó después de 2012?

Las ideas clave de la visión artificial (redes neuronales convolucionales y retropropagación) ya se entendían en 1990 y el algoritmo **LSTM** (memoria a corto y largo plazo) se desarrolló en 1997 y apenas ha cambiado. Pero hasta 2010, un computador de un terabyte era muy difícil de conseguir.

Tres nuevas figuras han impulsado el avance del aprendizaje automático:

1. Hardware más potente (GPU, TPU).
2. Mayor disponibilidad de datos (big data).
3. Avances en los algoritmos (mejores funciones de activación, optimizadores).

1.4.1 Machine Learning vs Deep Learning

Cuadro 1.5: Comparación entre Machine Learning y Deep Learning.

Criterio	Machine Learning	Deep Learning
Extracción de variables	Manual, mediante ingeniería de características	Automática, mediante capas de representación
Datos	Principalmente estructurados	Imágenes, texto, voz y otros no estructurados
Intervención manual	Requiere decirle si aprendió bien o mal	Aprende sin intervención manual
Escalabilidad	Computacionalmente eficiente, pero no se adapta bien a grandes volúmenes	Suele ser más eficaz en problemas complejos y grandes volúmenes
Costo computacional	Menor	Mayor
Interpretabilidad	Mayor	Menor

Advertencia 1.2:

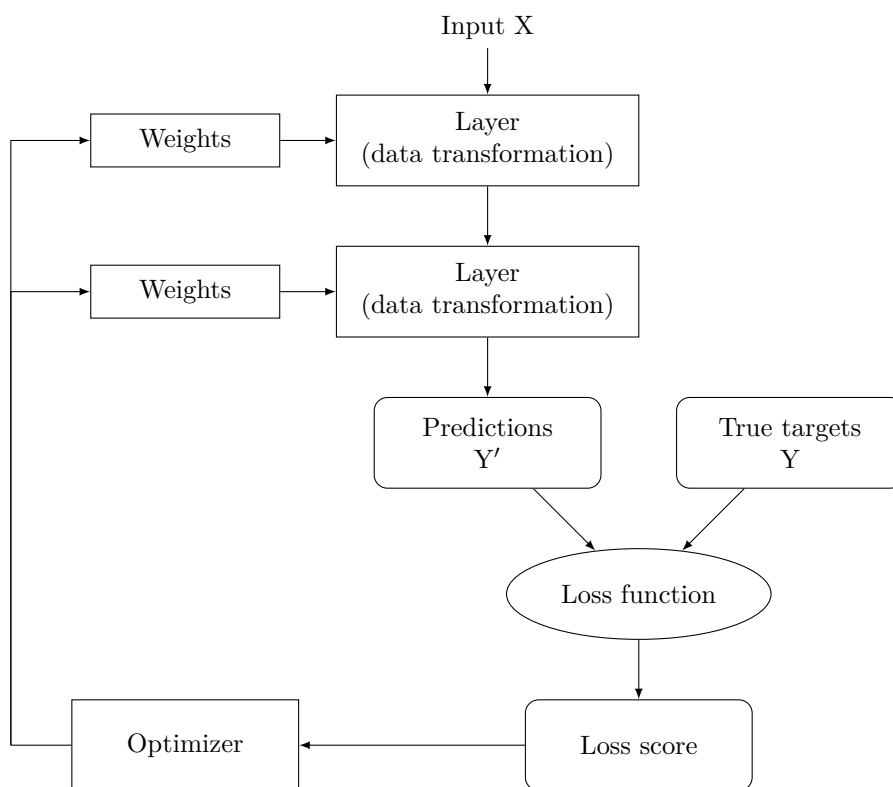
El Deep Learning es más caro y menos interpretable, pero más eficaz en problemas complejos.

1.4.2 Cómo trabaja Deep Learning

La formalidad y rigurosidad matemática detrás de los algoritmos de DL se verán más adelante. Por ahora, una explicación intuitiva:

El aprendizaje automático consiste en mapear *entradas* (como imágenes) a *objetivos* (como la etiqueta “gato”), procesando muchos ejemplos de entradas-objetivos. En Deep Learning, este mapeo se realiza a través de una secuencia de *transformaciones de datos simples*, que se aprenden mediante el procesamiento de muchos (realmente muchos) ejemplos.

- El cómo funciona cada capa, es decir, la especificación de lo que hace cada una con sus datos de entrada, se almacena en lo que se denomina **pesos de la capa**.
- En este contexto, **aprender** significa encontrar un conjunto de valores para los pesos de todas las capas de una red, tales que la red mapee correctamente las entradas iniciales a sus objetivos.
- Para controlar el resultado de una red, son necesarias mediciones, en particular, **qué tan lejos está el resultado de lo esperado**. Esta es la **función de pérdida** de la red (también llamada función objetivo o de costo), la cual toma las predicciones (Y') y el objetivo real (Y) calculando un puntaje de distancia (*loss score*).
- Esta puntuación se utiliza como **señal de retroalimentación para ajustar el valor de los pesos**, en una dirección que ayude a minimizar la puntuación de pérdida para el ejemplo actual.
- Este ajuste es tarea del **optimizador**, que implementa lo que se denomina **algoritmo de retropropagación** (*backpropagation*), el algoritmo central en el aprendizaje profundo.



1.5 Aplicaciones del Deep Learning

1.5.1 Visión por computadora (Computer Vision)

Aplicaciones cotidianas: Google Photos, búsqueda de imágenes de Google, YouTube, filtros de video en aplicaciones de cámara, software de OCR.

- **Galerías de fotos:** crean carpetas donde se identifican sujetos específicos.

- **Conducción autónoma:** un Tesla identificó mayor riesgo de atropellar a una persona y decidió chocar contra otro auto.
- **Agricultura autónoma:** drones con IA para optimizar el riesgo de cultivos.
- **Diagnóstico médico asistido por IA y sistemas de pago autónomos.**

1.5.2 Procesamiento de Lenguaje Natural (NLP)

En el lenguaje natural se contemplan idiomas humanos, ambiguos y cambiantes, como el español o el chino mandarín. Se distinguen de los lenguajes de máquina, estructurados cuidadosamente, como Assembly, Lisp, XML, etc.

Definición 1.7:

Natural Language Processing (NLP): uso de aprendizaje automático y grandes conjuntos de datos para procesar fragmentos de lenguaje y devolver algo útil.

Notar que el objetivo *no* es que la computadora “comprenda” el lenguaje, sino que, como dice la definición, devuelva algo útil.

Cuadro 1.6: Tareas comunes en NLP.

Tarea	Pregunta que responde
Clasificación de texto	¿Cuál es el tema de este texto?
Filtrado de contenido	¿Este texto contiene abuso?
Análisis de sentimientos	¿Este texto suena positivo o negativo?
Modelado del lenguaje	¿Cuál debería ser la siguiente palabra?
Traducción	¿Cómo dirías esto en alemán?
Resumen	¿Cómo resumirías este artículo en un párrafo?

1.5.3 Series temporales

Definición 1.8:

LSTM (Long short-term memory): algoritmo de memoria a corto y largo plazo, fundamental para el aprendizaje profundo en series temporales. Se desarrolló en 1997 y apenas ha cambiado desde entonces.

Definición 1.9:

Serie temporal: dato obtenido mediante mediciones en intervalos de tiempo regulares.

Ejemplos: precio de una acción, consumo eléctrico por hora, ventas semanales, clima, actividad sísmica.

Cuadro 1.7: Tareas sobre series temporales.

Tarea	Descripción	Ejemplo
Predicción	Anticipar lo que sucederá a continuación	Pronosticar consumo eléctrico con horas de anticipación
Clasificación	Asignar etiquetas categóricas a una serie	Clasificar si un visitante web es robot o humano
Detección de eventos	Identificar un evento esperado en un flujo continuo de datos	Detectar “Oye Siri” en audio

1.5.4 Deep Learning generativo y GANs

Definición 1.10:

Deep Learning generativo (IA generativa): crear nuevos contenidos realistas a partir del aprendizaje de distribuciones de datos.

Definición 1.11:

LLM (Large Language Model): modelo de lenguaje a gran escala; predice cuál es la siguiente palabra en una oración incompleta. Es la base de la IA generativa.

- **GANs** (redes generativas antagónicas): combinan una red generadora y una red discriminadora en un juego *minimax*, produciendo datos visuales indistinguibles de los reales.
- **VAEs** (variational autoencoders): generan datos mediante codificación y decodificación probabilística.
- **Modelos autoregresivos**: generan secuencias paso a paso (ej. Transformers).
- **ChatGPT**: mezcla LLM como generación de texto y usa mecanismos de autoatención. Los modelos de autoatención dominan la generación de secuencias, sobre todo texto.

1.6 Ética, sesgo y regulación

1.6.1 Sesgos discriminatorios

Los datos pueden ser falsos, imprecisos o poco representativos; aun así, se extrapolan. Los algoritmos de Machine Learning aprenden con datos históricos, por lo que se corre el riesgo de *perpetuar los prejuicios ya instalados* en los datos del pasado.

Advertencia 1.3:

Los datos de entrenamiento pueden contener sesgos; si no se corrigen, el modelo los reproduce o amplifica.

Ejemplo: Estudios sobre modelos que analizan la rotación de camas UCI en una clínica privada chilena permiten ver sesgos y evaluar si son aplicables a la población general de Chile.

1.6.2 Privacidad y regulación

Las organizaciones han creado **comités de ética** para evitar el uso inadecuado de IA. Un aspecto clave es la explicabilidad: la organización que usa IA debe explicar el porqué de sus resultados.

Eventualmente se crea el **GDPR** (Reglamento General de Protección de Datos de Europa).

Implicancias:

- Si un banco niega un préstamo mediante IA, debe explicar por qué.
- Los algoritmos de IA no pueden utilizar datos personales durante su fase de aprendizaje.

En Chile se está promulgando la **Ley 21.790** para la protección de datos personales, que tiene como base el GDPR europeo.

Capítulo 2

El Perceptrón y Redes Fully Connected Feed Forward

2.1 De la IA a las redes neuronales

Una red se puede pensar como una malla de nodos entrelazados con diferentes uniones. En Deep Learning programamos un sistema que en cada capa decide hacia qué nodo avanzar según una probabilidad dada por un algoritmo.

En **Representation Learning** (aprendizaje de representaciones) se construyen representaciones de las entradas o inputs en etapas sucesivas. En cada etapa, la representación o abstracción de la anterior contiene información más compacta y relevante, pero en menor cantidad. Eventualmente, con esta abstracción sucesiva se crea un tensor. Porque transforman la entrada inicial hasta generar un tensor o vector final que contiene todo lo necesario para tomar una decisión, estos algoritmos son llamados **End-to-End**.

La diferencia fundamental entre Machine Learning y Deep Learning radica en la *extracción de características*:

Cuadro 2.1: Extracción de características: ML vs DL.

Enfoque	Proceso
Machine Learning	Extracción manual de características → clasificador → salida
Deep Learning	Entrada → red neuronal → salida, sin intervención manual

Rendimiento, datos y costo computacional: A diferencia de los modelos tradicionales de Machine Learning (cuyo rendimiento se satura rápidamente al aumentar los datos), los modelos de Deep Learning escalan a medida que se incrementan la cantidad de datos, el número de capas y la capacidad computacional. La idea es que se *podría* resolver prácticamente cualquier problema, dados una cantidad de datos y capacidad de cómputo suficientes, siendo estos mismos el factor limitante.

2.2 El perceptrón

El **perceptrón** se inspira en el funcionamiento de la red neuronal biológica del cerebro humano. En este, cada neurona recibe impulsos eléctricos de neuronas vecinas. Cuando la cantidad total de las señales entrantes supera cierto **umbral**, la neurona “dispara” su propia señal, la cual se conecta con otras neuronas.

La unión o punto de contacto entre dos neuronas se llama **sinapsis**. Cuando aprendemos nuevas asociaciones entre dos conceptos, la **fuerza sináptica** entre las neuronas que representan dichos conceptos se fortalece.

Regla de Hebb (1949): “Las células que se **activan juntas**, se conectan entre sí”.

Definición 2.1:

Perceptrón (Rosenblatt, 1957): modelo simplificado de una neurona biológica llevado al computador.

2.2.1 Operación del perceptrón

Intuición El perceptrón es la unidad básica de una red neuronal. Recibe un conjunto de m señales de entradas binarias x_i (por ejemplo, píxeles encendidos o apagados), cada una con un cierto nivel de **importancia** (los pesos w_i), y un **umbral de activación** ajustable (el sesgo o bias b). El perceptrón suma todas las señales *ponderadas* y, si el total supera cierto umbral, se “activa” y devuelve 1; en caso contrario, devuelve 0. Podría decirse que este proceso de decisión binaria lo convierte en un clasificador. *El diagrama de la Figura 2.1 ilustra estos componentes, los cuales se definen formalmente a continuación.*

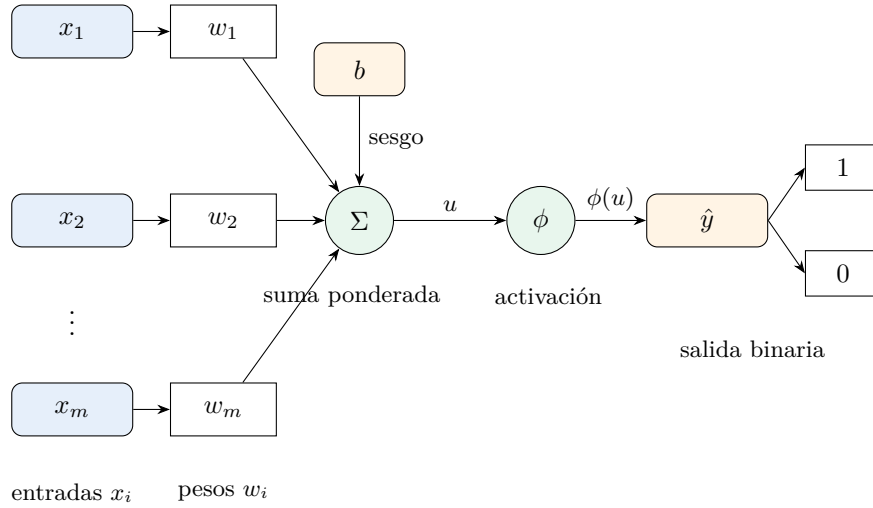


Figura 2.1: Diagrama del perceptrón: suma ponderada de las entradas y función de activación.

Definición formal Más rigurosamente, entendemos al perceptrón como una función compuesta por una función de combinación lineal o suma ponderada ($u(\vec{x}, \vec{w}, b)$) y una función de activación escalón ($\phi(u)$).

El perceptrón recibe tres argumentos: un vector columna de entradas binarias $\vec{x}$, un vector columna de pesos $\vec{w}$ y un número real o escalar b , llamado sesgo. La función, tras procesar las entradas, devuelve un valor binario $\hat{y}$, que indica si este fue activado o no. Se detallan a continuación:

- $\vec{x} = (x_1, x_2, \dots, x_m)^\top \in \{0, 1\}^m$ es el vector columna de entradas binarias (características). Los valores x_i solo pueden ser 0 o 1, y m es la cantidad de dimensiones del vector, es decir, la cantidad de entradas.
- $\vec{w} = (w_1, w_2, \dots, w_m)^\top \in \mathbb{R}^m$ es el vector columna de **pesos** (*weights*), que ponderan o escalan la importancia de cada entrada binaria. Los valores w_i son números reales, y la dimensión m coincide con la de $\vec{x}$, condición necesaria para que el producto escalar $\vec{w}^\top \vec{x}$ esté definido.
- $b \in \mathbb{R}$ es el **sesgo** (*bias*), un parámetro de desplazamiento u *offset* que ajusta el umbral de activación de la neurona.
- $\hat{y} \in \{0, 1\}$ es el resultado binario final de la función, que indica la activación (o no) del perceptrón.

Así, queda definido el dominio de la función del perceptrón, y su proceso se detalla a continuación:

$$P : \{0, 1\}^m \times \mathbb{R}^m \times \mathbb{R} \rightarrow \{0, 1\}$$

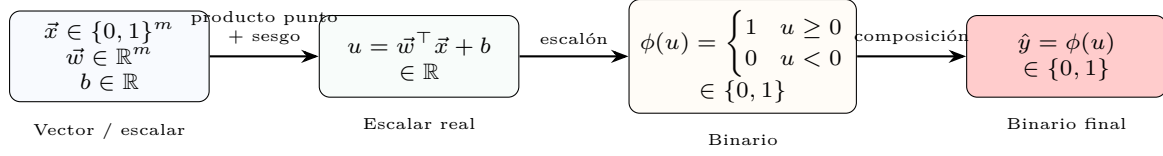


Figura 2.2: Recorrido de las variables en el perceptrón.

Paso 1: Entrada neta Primero, se calcula la suma *ponderada* de las entradas más el sesgo. Esta operación pondera los vectores $\vec{x}$ y $\vec{w}$, cuyo producto escalar las combina en un número real, y le suma el sesgo b . Formalmente:

$$u : \{0, 1\}^m \times \mathbb{R}^m \times \mathbb{R} \rightarrow \mathbb{R}$$

$$u(\vec{x}, \vec{w}, b) = \vec{w}^\top \vec{x} + b = \sum_{i=1}^m w_i x_i + b = w_1 x_1 + w_2 x_2 + \cdots + w_m x_m + b \quad (2.1)$$

Nota sobre la notación: Dado que los vectores $\vec{x}$ y $\vec{w}$ se definen como vectores columna, la transpuesta $\vec{w}^\top$ convierte a $\vec{w}$ en un vector fila para que el producto $\vec{w}^\top \vec{x}$ (fila por columna) sea un escalar, que corresponde a la suma $\sum_{i=1}^m w_i x_i$.

Nota sobre variantes del diagrama: El diagrama muestra la representación **directa** del sesgo, donde b se suma como término independiente: $u = \sum_{i=1}^m w_i x_i + b$. Existe una variante equivalente, que añade una entrada ficticia $x_0 = 1$ y un peso $w_0 = b$, unificando la operación en una única suma ponderada:

$$u = \sum_{i=0}^m w_i x_i = w_0 \cdot 1 + w_1 x_1 + \cdots + w_m x_m = b + w_1 x_1 + \cdots + w_m x_m.$$

Su utilidad es principalmente **algebraica**: permite expresar el perceptrón como un único producto punto $\vec{w}^\top \vec{x}$, simplificando derivadas, reglas de aprendizaje y demostraciones teóricas. Ambas representaciones son matemáticamente equivalentes y su elección es contextual.

Paso 2: Función de activación El valor resultante u (que es un número real) se transforma en una salida binaria, que es 1 si $u \geq 0$, y 0 si $u < 0$. Esto se logra mediante la función de activación escalón unitario, la cual introduce la no linealidad y toma la decisión final de activación:

$$\phi : \mathbb{R} \rightarrow \{0, 1\}$$

$$\phi(u) = \begin{cases} 1 & \text{si } u \geq 0, \\ 0 & \text{si } u < 0. \end{cases} \quad (2.2)$$

La Figura 2.3 muestra su forma: para cualquier valor negativo de u , la salida es 0; para u igual o mayor que cero, la salida es 1.

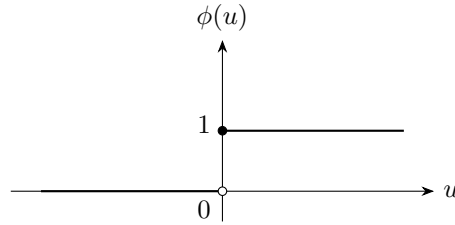


Figura 2.3: Función de activación escalón unitario.

Notar que más adelante, la función escalón se vuelve obsoleta, y eventualmente es reemplazada por la función ReLU.

Paso 3: Salida final del perceptrón Finalmente, la salida del modelo es la composición de la combinación lineal afín (suma ponderada) y la función de activación:

$$\hat{y} = P(\vec{x}, \vec{w}, b) = \phi(u(\vec{x}, \vec{w}, b)) = \phi(\vec{w}^\top \vec{x} + b). \quad (2.3)$$

En resumen, el perceptrón transforma un vector de entradas binarias $\vec{x}$ en una salida binaria $\hat{y}$ mediante una suma ponderada ajustada por un sesgo y una decisión de umbral. Los parámetros del modelo, $\vec{w}$ y b , son los que se ajustan durante el entrenamiento para que la salida $\hat{y}$ se aproxime al valor deseado para cada ejemplo de entrada.

2.2.2 Funciones de activación

Una función de activación es una función no lineal que transforma la entrada neta u de una neurona en una señal de salida. Biológicamente, intenta emular el comportamiento de una neurona real: cuando la acumulación de estimulación supera un cierto umbral, la neurona “dispara” información hacia la siguiente neurona.

En las funciones de activación *siempre* se deben utilizar funciones **no lineales**. Si se usaran funciones lineales, la composición de múltiples capas sería equivalente a una única transformación lineal, anulando la capacidad de la red para aprender patrones complejos.

A continuación se estudian las cuatro funciones de activación más relevantes: la función escalón (histórica), la sigmoide, la tangente hiperbólica (tanh) y ReLU.

Originalmente, Rosenblatt utilizó la función escalón en su perceptrón:

Definición 2.2:

Función escalón (step)

$$\phi(u) = \begin{cases} 0 & \text{si } u < 0, \\ 1 & \text{si } u \geq 0. \end{cases} \quad (2.4)$$

Esta función *no es utilizable* en el entrenamiento moderno. Para entrenar una red neuronal necesitamos calcular derivadas (gradientes) mediante retropropagación, y la derivada de esta función es 0 en todos los puntos donde está definida (es una función constante por tramos). Esto impide que los pesos se actualicen.

La función que resolvió el problema de la derivabilidad es la **sigmoide logística**

Definición 2.3:

Función sigmoide

$$\sigma(u) = \frac{1}{1 + e^{-u}}. \quad (2.5)$$

Es derivable en todo $\mathbb{R}$ y su rango es $(0, 1)$, lo que la hace adecuada para interpretar salidas como probabilidades. Emula razonablemente bien el comportamiento de una neurona biológica.

Su principal desventaja es la **saturación en los extremos**: para valores de u muy grandes o muy pequeños, la pendiente (gradiente) es prácticamente 0, lo que provoca que los gradientes desaparezcan durante la retropropagación y la red deje de aprender.

Definición 2.4:

Función tangente hiperbólica (tanh)

$$\tanh(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}. \quad (2.6)$$

Su rango es $(-1, 1)$, por lo que está centrada en cero, lo que facilita el entrenamiento. Funciona mejor que la sigmoide en la práctica.

Presenta el mismo problema de saturación en los extremos: el gradiente se vuelve muy pequeño para valores grandes de $|u|$, lo que puede ralentizar o detener el aprendizaje.

Definición 2.5:

Función ReLU (Rectified Linear Unit)

$$\text{ReLU}(u) = \begin{cases} 0 & \text{si } u < 0, \\ u & \text{si } u \geq 0. \end{cases} \quad (2.7)$$

Es la función de activación más utilizada en la práctica actual. Para valores positivos, el gradiente es 1 y *nunca desaparece*, lo que acelera el entrenamiento significativamente.

Regla de uso: ReLU se utiliza en las *capas ocultas* (intermedias). *No se usa en la capa de salida* para clasificación; en su lugar se emplea softmax. Tampoco se recomienda usar ReLU cuando los datos de entrada pueden tomar valores negativos muy grandes, ya que las neuronas pueden “morir” (quedar atascadas en 0).

Cuadro 2.2: Comparación de las funciones de activación.

Función	Rango	Ventajas	Desventajas
Escalón	$\{0, 1\}$	Histórica, simple	No derivable, gradiente cero
Sigmoide	$(0, 1)$	Derivable, interpretable	Saturación en extremos
Tanh	$(-1, 1)$	Centrada en cero	Saturación en extremos
ReLU	$[0, \infty)$	No satura para positivos, rápida	Neuronas “muertas” para negativos

2.3 Arquitectura de una red neuronal

2.3.1 Multi-perceptrón

Limitación del perceptrón simple: El perceptrón simple (una sola neurona) solo puede aprender funciones linealmente separables, es decir, problemas donde los datos se pueden dividir por una línea recta. Para problemas más complejos, necesitamos redes con múltiples neuronas organizadas en capas: el **multi-perceptrón** o *red neuronal multicapa*.

La realidad requiere extender el perceptrón a más dimensiones. En una red multicapa, el vector de entrada ya no se conecta a una única neurona, sino a *muchas neuronas* en paralelo. Esto da lugar a una **matriz de pesos** (no un solo vector de pesos). Es decir, pasamos de $\vec{w} \in \mathbb{R}^m$ a $\vec{W}^{(i)} \in \mathbb{R}^{F_{i-1} \times F_i}$.

Componentes de una red neuronal multicapa

1. **Capa de entrada:** contiene los datos de entrada. No es una capa en el sentido estricto, ya que no realiza ninguna transformación; simplemente distribuye los datos a la siguiente capa.

2. **Capas ocultas (escondidas):** son las capas intermedias donde ocurren las transformaciones no lineales de los datos. La red aprende representaciones cada vez más abstractas a medida que avanzan las capas.
3. **Capa de salida:** entrega el resultado final de la red. Para problemas de clasificación, suele ser un vector de probabilidades (usando softmax); para regresión, un escalar o vector de valores continuos.

Es importante notar que al aumentar la cantidad de neuronas por capa y/o el número de capas de la red, aumentamos drásticamente la cantidad de **parámetros** (pesos y sesgos), lo que incrementa el tiempo de entrenamiento y la demanda de memoria.

Tipos de redes neuronales

- **Red neuronal profunda (deep neural network):** una red neuronal que tiene *más de una capa oculta*. Cuantas más capas tiene, más profunda se considera.
- **Fully Connected (completamente conectada):** una arquitectura donde todas las neuronas de una capa están conectadas con todas las neuronas de la siguiente capa.
- **Feed Forward (hacia adelante):** la información fluye siempre en una dirección, desde la entrada hasta la salida, sin ciclos ni retroalimentación.

2.3.2 Redes Feed Forward Fully Connected (FFFC)

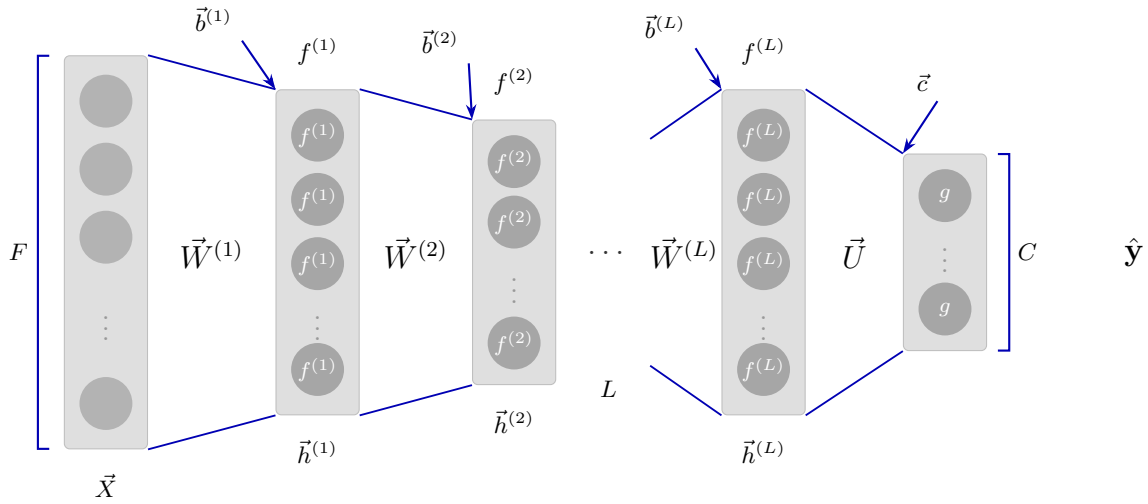


Figura 2.4: Arquitectura de una red Feed Forward.

Capa de entrada

- $\vec{X} \in \mathbb{R}^{n \times F}$: Matriz de entrada (datos), donde n es el tamaño del lote (*batch size*) y F es el número de características de entrada. Para un solo ejemplo, $\vec{X} \in \mathbb{R}^{1 \times F}$.
- F : Número de neuronas (características) de la capa de entrada.

Capas ocultas ($i = 1, \dots, L$)

- L : Número total de capas ocultas.

- $\vec{W}^{(i)} \in \mathbb{R}^{F_{i-1} \times F_i}$: Matriz de pesos de la capa oculta i , donde F_{i-1} es el número de neuronas de la capa anterior y F_i el de la capa actual.
- $\vec{b}^{(i)} \in \mathbb{R}^{1 \times F_i}$: Vector fila de sesgos de la capa oculta i .
- $f^{(i)} : \mathbb{R} \rightarrow \mathbb{R}$: Función de activación (aplicada elemento a elemento) de la capa oculta i .
- $\vec{h}^{(i)} \in \mathbb{R}^{n \times F_i}$: Matriz de salida de la capa oculta i (o vector fila $\mathbb{R}^{1 \times F_i}$ para un solo ejemplo).

Capa de salida

- $\vec{U} \in \mathbb{R}^{F_L \times C}$: Matriz de pesos de la capa de salida.
- $\vec{c} \in \mathbb{R}^{1 \times C}$: Vector fila de sesgos de la capa de salida.
- $g: \mathbb{R} \rightarrow \mathbb{R}$: Función de salida (aplicada elemento a elemento), como softmax o identidad.
- $\hat{\mathbf{y}} \in \mathbb{R}^{n \times C}$: Matriz de salida final de la red (o vector fila $\mathbb{R}^{1 \times C}$ para un solo ejemplo), que contiene las predicciones.
- C : Número de neuronas de la capa de salida (número de clases para clasificación, o 1 para regresión).

Fórmula de la capa oculta Para cada capa oculta i , la salida de tal capa se calcula como:

$$\vec{h}^{(i)} = f^{(i)}\left(\vec{h}^{(i-1)} \vec{W}^{(i)} + \vec{b}^{(i)}\right) \quad (2.8)$$

donde asumimos que $\vec{h}^{(0)} = \vec{X}$, es decir, la entrada es la capa 0.

La multiplicación $\vec{h}^{(i-1)} \vec{W}^{(i)}$ es una operación matricial donde cada fila de $\vec{h}^{(i-1)}$ se multiplica por la matriz de pesos. El resultado se suma al vector de sesgos $\vec{b}^{(i)}$, y luego se aplica la función de activación $f^{(i)}$ elemento a elemento.

Sobre la notación: Acá los vectores se representan como **filas** y las matrices de pesos se multiplican por la derecha ($\vec{h} \vec{W}$). Esto difiere de la notación de vectores columna utilizada en el perceptrón simple ($\vec{w}^\top \vec{x}$), pero ambas son matemáticamente equivalentes.

Fórmula de la capa de salida La salida de la red se calcula como:

$$\hat{\mathbf{y}} = g\left(\vec{h}^{(L)} \vec{U} + \vec{c}\right), \quad (2.9)$$

donde L es el número total de capas ocultas, y $\hat{\mathbf{y}}$ es la salida final de la red.

Aspectos técnicos de las operaciones matriciales En la práctica, para aprovechar el hardware moderno (GPUs, TPUs), los datos se procesan en **lotes** (*batches*) en lugar de ejemplo por ejemplo. Esto significa que $\vec{X}$ ya no es un vector de tamaño $(F, 1)$, sino una **matriz de tamaño** (n, F) , donde n es el tamaño del lote (*batch size*) y F es el número de características de entrada. De manera análoga, la salida $\hat{\mathbf{y}}$ es una matriz de tamaño (n, C) .

Las librerías modernas como TensorFlow y PyTorch están altamente optimizadas para operaciones matriciales en GPU, lo que acelera drásticamente los cálculos.

2.3.3 Función de salida: Softmax

Para problemas de clasificación multiclase, se utiliza la función **softmax** en la capa de salida.

Definición 2.6:

Softmax: función de salida que transforma un vector de valores reales en una *distribución de probabilidades* sobre las clases. Para la componente j del vector de salida:

$$s_j = \frac{e^{z_j}}{\sum_{k=1}^C e^{z_k}}, \quad (2.10)$$

donde C es el número de clases y z_j es el valor de la neurona j antes de aplicar softmax.

Propiedades:

- Cada s_j esté en el intervalo $(0, 1)$.
- La suma de todos los s_j sea 1: $\sum_{j=1}^C s_j = 1$.

Esto permite interpretar la salida como una distribución de probabilidad sobre las clases.

Ejemplo: Clasificación de estados de ánimo (4 clases): pena, alegría, depresión, ansiedad. La salida de la red es un vector de cuatro neuronas. Si softmax asigna 0.90 a la clase “alegría”, significa que el modelo predice con un 90% de probabilidad que el estado de ánimo es alegría.

Advertencia 2.1:

Es importante no confundir la **función de activación** (usada en capas ocultas, generalmente ReLU) con la **función de salida** (usada en la capa de salida, como softmax o identidad). Incluso una función como sigmoide, que no se recomienda como activación en capas ocultas, puede utilizarse como función de salida para problemas de clasificación binaria.

2.3.4 Entrenamiento de la red neuronal

Definición 2.7:

Entrenamiento: el proceso mediante el cual la red encuentra los valores óptimos para todos sus parámetros (pesos y sesgos) de modo que resuelva el problema de forma satisfactoria.

Para entrenar, se inicializan los pesos con valores aleatorios pequeños (centrados en cero) y se itera entre la primera capa oculta y la capa de salida (el **forward pass**), ajustando los parámetros mediante retropropagación.

Datos y etiquetas Para entrenar una red se necesitan:

- $\vec{y}$ (sin sombrero): la **etiqueta** correcta para cada dato de entrada (el valor esperado).
- $\hat{\mathbf{y}}$ (con sombrero): la **predicción** que calcula la red durante el entrenamiento.

El objetivo es que $\hat{\mathbf{y}} \approx \vec{y}$.

Función de error y pérdida (loss)

Para cuantificar el error, se define una **función de pérdida** que mide la discrepancia entre la salida esperada $\vec{y}$ y la predicción $\hat{\mathbf{y}}$.

Definición 2.8:

Función de pérdida (*loss function*): función que mide la diferencia entre la salida esperada $\vec{y}$ y la salida calculada $\hat{\mathbf{y}}$, devolviendo un escalar que indica qué tan mal se está desempeñando el modelo en un ejemplo o en un lote.

Para un conjunto de n ejemplos, la pérdida promedio es:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n \ell(\hat{\mathbf{y}}^{(i)}, \vec{y}^{(i)}), \quad (2.11)$$

donde ℓ es la función de error para un solo ejemplo (por ejemplo, error cuadrático medio o entropía cruzada).

Dependencia de los parámetros La función de pérdida depende de *todos* los parámetros de la red:

$$\Theta = \left\{ \vec{W}^{(i)}, \vec{b}^{(i)}, \vec{U}, \vec{c} \right\}_{i=1}^L \quad (2.12)$$

El objetivo del entrenamiento es encontrar el conjunto de parámetros Θ que **minimiza** la función de pérdida:

Este problema de minimización se resuelve mediante el algoritmo de **descenso del gradiente** y su variante *backpropagation*, que calcula los gradientes de la pérdida con respecto a cada parámetro.

Interpretación de la pérdida y exactitud

- La **pérdida** debe ser *cercana a 0*, pero no exactamente 0.
- Si la pérdida es exactamente 0, el modelo probablemente *memorizó* el conjunto de entrenamiento y no generalizará bien a datos nuevos (**sobreajuste**).
- La **exactitud** (*accuracy*) es el porcentaje de predicciones correctas sobre el total de ejemplos.
- Idealmente, la exactitud debe ser alta (por ejemplo, superior a 95%), pero inferior a 100% para asegurar generalización.